

Python Fundamentals Project

Project 1: Simple Port Scanner

Overview:

This project involves creating a port scanning tool used to probe a target system or network to identify which ports are open and listening for connections. This project introduces beginners to key cybersecurity concepts like networking, sockets, and TCP/UDP communication while strengthening Python skills.

Project Goals:

- Learn to use Python's *socket* module.
- Understand the basics of port scanning and network security.
- Develop a simple tool to scan for open ports on a target system.
- Understand the concept of open ports and why securing them is critical.

Features of the Port Scanner:

1. Accepts a target IP address or hostname.
2. Allows the user to specify a range of ports to scan.
3. Scans for open TCP or UDP ports.
4. Prints the results (open ports) in a user-friendly format.
5. Retrieve and display service banners for open ports.
6. Save scan results to a file for later analysis.
7. Add functionality to ping a range of IPs to discover live hosts.
8. Speed up scanning by checking multiple ports simultaneously (Multi-Threading).

Project 2: File Encryption and Decryption Tool

Overview:

This project involves creating a simple Python script to encrypt and decrypt files using a symmetric encryption algorithm. The project introduces beginners to cryptography basics, file handling, and Python's popular *cryptography* library.

Project Goals:

- Learn the basics of cryptography, including encryption and decryption.
- Understand symmetric encryption using a secret key.
- Develop skills in file handling in Python.
- Understand symmetric encryption and its applications in file security.
- Learn about the importance of secure key management in cybersecurity.

Features of the File Encryption-Decryption Tool:

1. Encrypt a file using a secret key.
2. Decrypt the encrypted file back to its original state using the same key.
3. Provide user-friendly prompts for selecting operations and files.
4. Generate and securely store the encryption key.
5. Protect the encryption key with a password.
6. Add support for encrypting or decrypting multiple files at once.
7. Implement secure key storage and retrieval.
8. Add a feature to verify the integrity of files using hashes.

Project 3: Packet Sniffing

Overview:

This project involves creating a tool that captures network packets in real-time and provides information such as source and destination IPs, protocols, and payloads. This project introduces beginners to network analysis, teaching them how to use Python for capturing and inspecting network traffic.

Project Goals:

- Learn about network packets and how they are structured.
- Use Python's *scapy* library to capture and analyze packets on a network interface.
- Understand the role of packet sniffers in cybersecurity, such as monitoring and troubleshooting.

Features of the Packet Sniffer:

1. Capture packets from a network interface.
2. Extract and display key information such as:
 - Source and destination IP addresses.
 - Protocols (e.g., TCP, UDP, ICMP).
 - Packet payloads.
3. Allow the user to filter packets by protocol or port.
4. Extract additional details, such as packet length, flags, or TTL.
5. Save captured packets to a .pcap file for later analysis using tools like Wireshark.

Project 4: Log Recon

Overview:

This project involves creating a tool to process and analyze log files (auth.log) from servers. This project teaches beginners how to parse text files, search for specific patterns, and generate meaningful insights from logs. It's practical for understanding log formats and gaining experience with regular expressions, file handling, and cybersecurity incident detection.

Project Goals:

- Learn to read and process large log files.
- Identify key events such as failed logins, successful logins, and unauthorized access attempts.
- Understand common log formats in auth.log
- Analyse potential threats by detecting suspicious behavior or attack patterns from logs.

Features of the Packet Sniffer:

1. Log Parse auth.log: Extract command usage.
 - Include the Timestamp.
 - Include the executing user.
 - Include the command.
2. Log Parse auth.log: Monitor user authentication changes.
 1. Print details of newly added users, including the Timestamp.
 2. Print details of deleted users, including the Timestamp.
 3. Print details of changing passwords, including the Timestamp.
 4. Print details of when users used the su command.
 5. Print details of users who used the sudo; include the command.
 6. Print ALERT! If users failed to use the sudo command; include the command.
3. Save the analysis to a summary file
4. Use a library like geopy to find geographic locations of IP addresses.
5. Enable analysis of multiple log files at once.